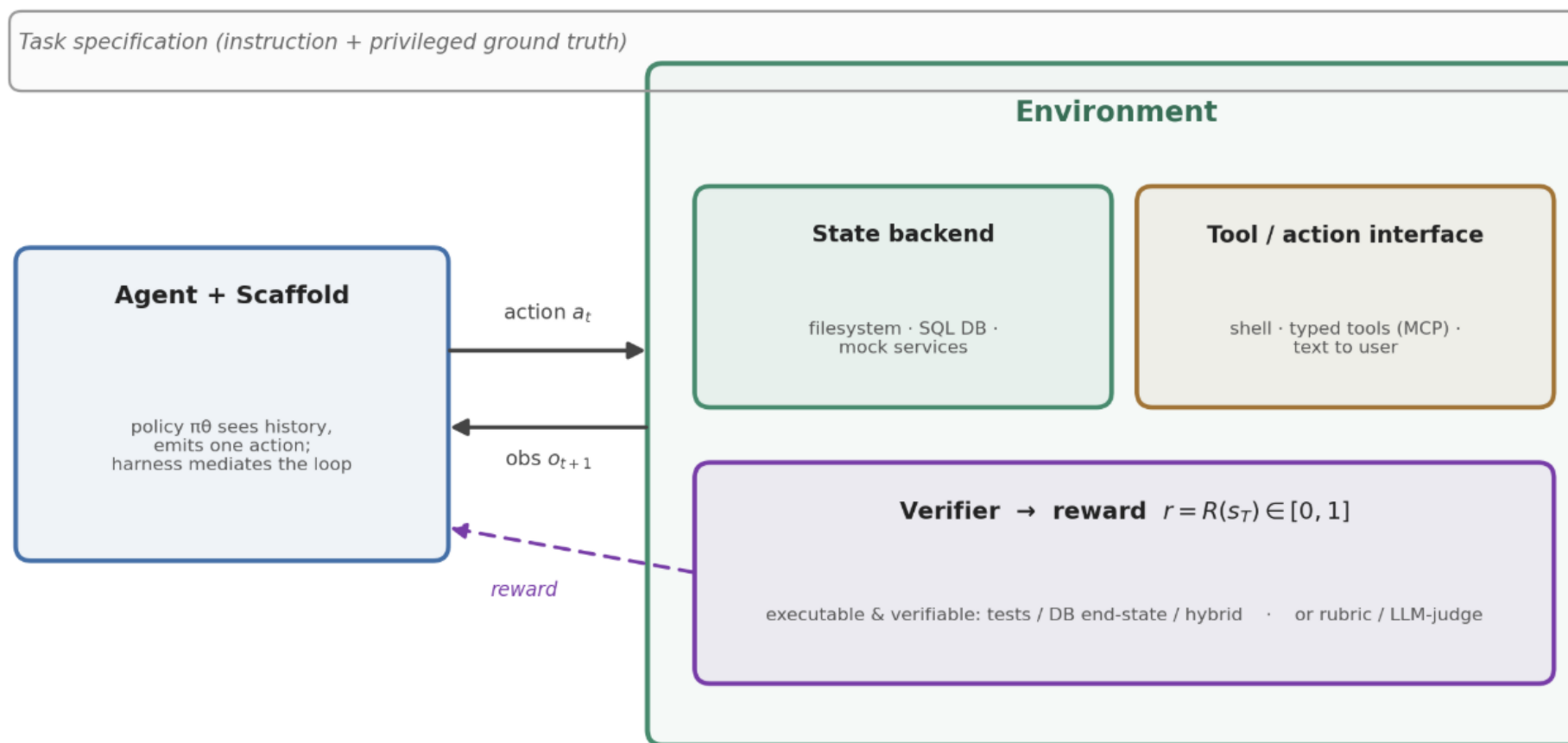


agent env

赵钰哲

- agentic bench为评测而建

但训练agent能力需要足够多的交互feedback
因此需要合成环境 用来：评测 训练

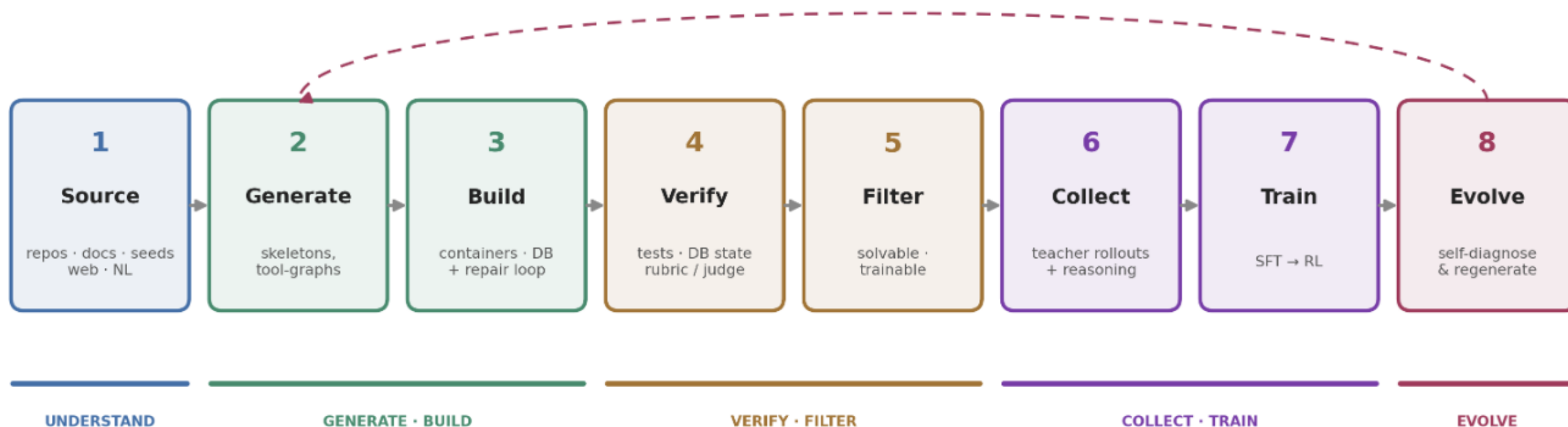


The five parts of any agent environment: task spec · state backend · tool interface · verifier/reward · agent scaffold.

生成环境的pipeline

The Environment-Scaling Pipeline

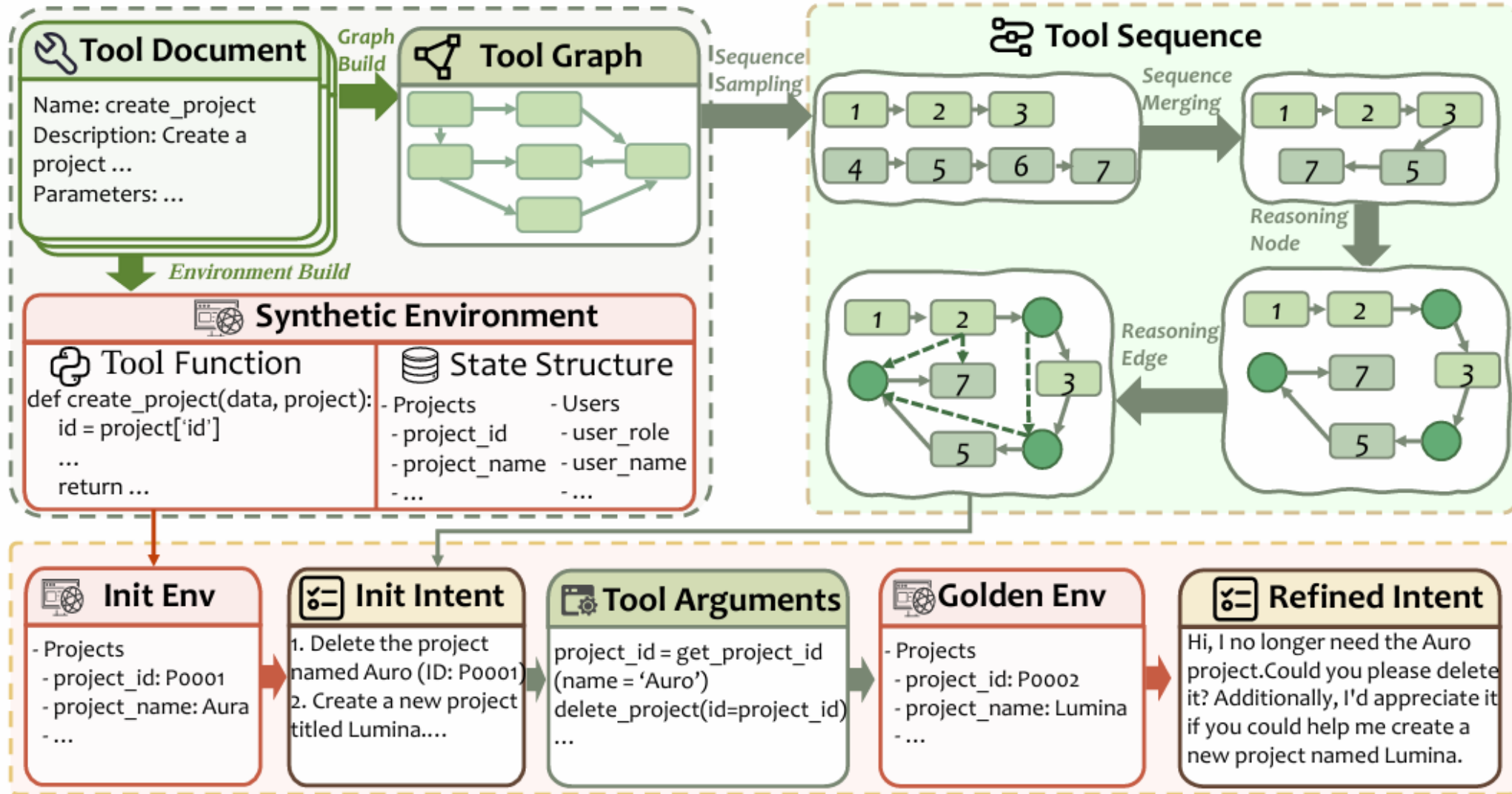
self-diagnosing feedback loop



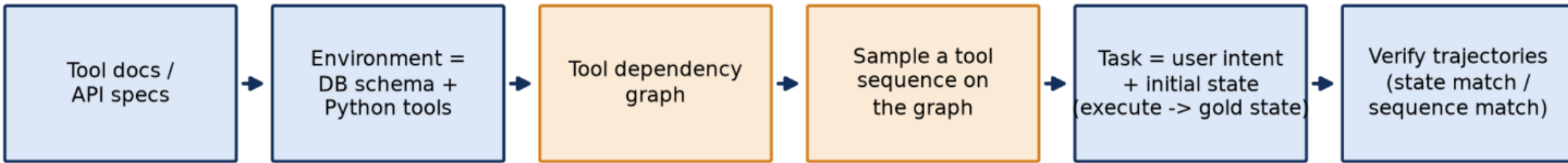
Generate

- 状态变量、规则、工具作为skeleton，在上面实例化具体任务
- 把工具看成一张图
 - 只要一个工具的输出能喂给另一个的输入就连一条边
 - 然后在tool-graph上做随机游走得到连贯的多步任务
- AutoForge
 - 在随机游走的序列里加入单向推理节点来提高难度

AutoForge



Shared skeleton of AgentScaler / AutoForge / EnvFactory (orange = where the three papers differ most)



Paper

How the graph edges are built

How a tool sequence is sampled

AgentScaler
2509.13311

edge (i, j) if cos-sim of the two parameter-list vectors > tau;
Louvain communities = domains (> 1,000); an LLM re-checks pairs inside a domain

directed walk from a random start node until a sink
or the max step count -> one linear chain

AutoForge
2512.22857

edge A -> B if an LLM judges
"the output of A is likely a valid input of B"

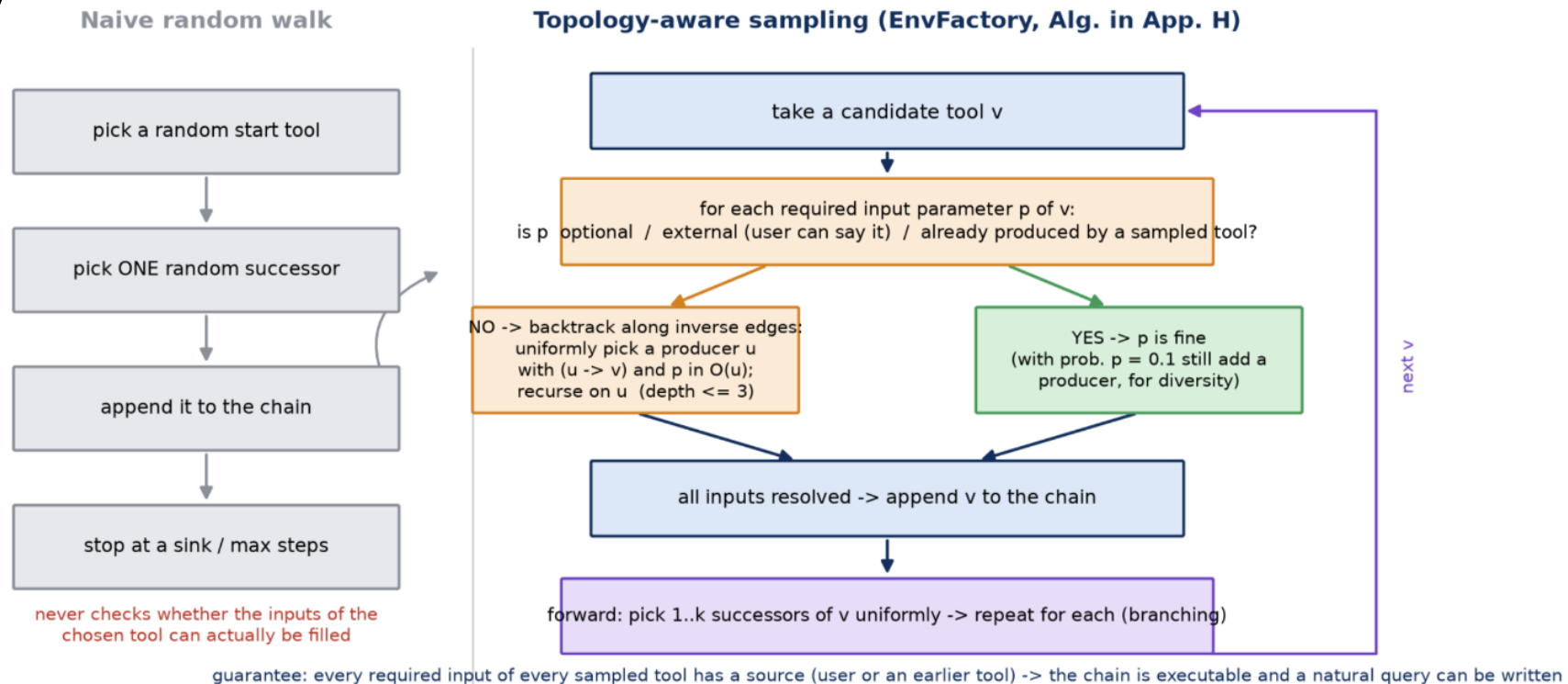
random walks -> merge two chains (drop redundant tools)
-> insert reasoning nodes -> add reasoning edges -> DAG

EnvFactory
2605.18703

tools AND parameters are nodes; edge A -> B if any output param of A
matches an input param of B (bge-m3, cos > 0.85); LLM adds / prunes edges per env

topology-aware: before adding a tool, resolve every required
input backwards (depth <= 3); then branch to 1..k successors

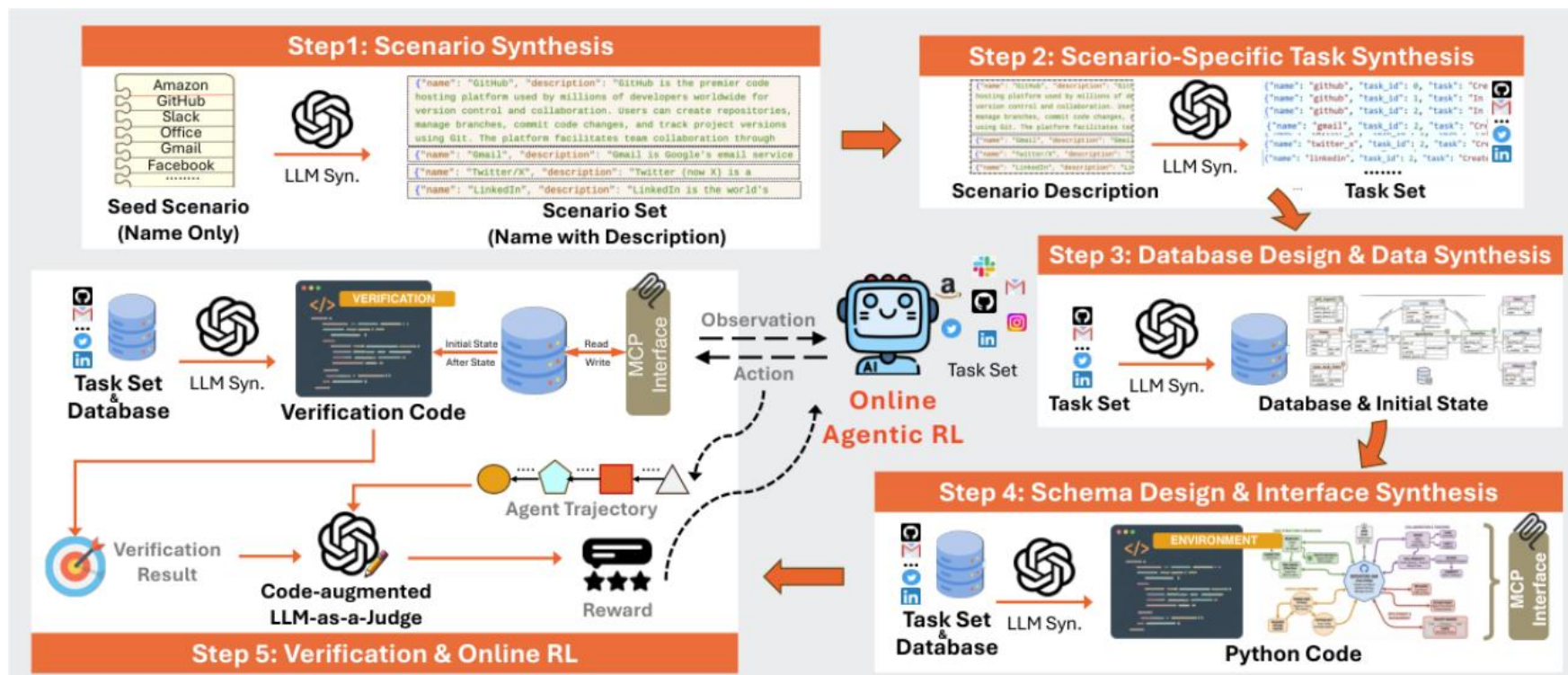
EnvFactory



- 在选取某个工具前先解析它的输入依赖，产出合法的非线性链
如果输入有依赖，就补齐再继续

Build

- 产出 容器 数据库schema 种子数据 可执行的工具 初始状态检查
- LLM带修复循环：生成代码、尝试构建并运行、把任何构建测试报错喂回给模型、最多重试k次；永远构建不起来的就丢掉



环境里任务的Verify

- 执行测试 — 针对文件、进程、输出的断言
- 数据库末态比对

把后端存储与预期末态做 diff [AgentScaler](#)、AWM

- Hybrid verifier

把跑测试与一个读取轨迹的学习型模型结合起来

对 test-time 选择很有用 [R2E-Gym](#)

- Rubric / LLM-as-judge

generator-verifier 不对称

容易验证的领域往往难以生成好任务，反之亦然

对合成的环境 filter

- solvable 强模型能取得非零成功率 $\text{pass@16} > 0$
- non-trivial 不是所有模型都能一次答对
- non-contaminated 训练任务不是评测 benchmark 的近似重复
- trainable 能产生可用梯度
- 丢弃太多会增加损耗

可以采用AWM的修复循环

让 $>85\%$ 的环境一次构建成功（平均约 1.1 次尝试）

collect轨迹 & 训练

- 强teacher对每个任务多次解答尝试 保留成功的轨迹
轨迹作dual-use 同时用于SFT和RL
- SFT warm start
指令遵循，经过SFT后模型应输出scaffold 的动作/观测格式
使模型能产出成功rollout以获得RL训练梯度
有SFT学会scaffold会好得多，RL冷启动几乎无效
- RL
PPO GRPO
关键在防止一个抖动的被模拟用户或一个坏环境主导更新
(例如屏蔽那些因被模拟用户行为不当而失败的 episode)

Rollout Pass-Rate Control: Steering Binary-Reward RL Toward Its Most Informative Regime

Tianshu Zhu¹, Wenyu Zhang¹, Xiaoying Zuo¹, Lun Tian¹, Haotian Zhao¹,
Yucheng Zeng, Jingnan Gu^{1†}, Daxiang Dong^{1†}, Jianmin Wu^{1†},
Dawei Yin^{1†}, Dou Shen^{1†}

¹Baidu

Emails: {zhutianshu, zhangwenyu08, zuoxiaoying, tianlun, zhaohaotian02,
gujingnan, dongdaxiang, wujianmin, yindawei02, shendou}@baidu.com

[†]Corresponding authors: Jingnan Gu, Daxiang Dong, Jianmin Wu, Dawei Yin, and Dou Shen

四个判据

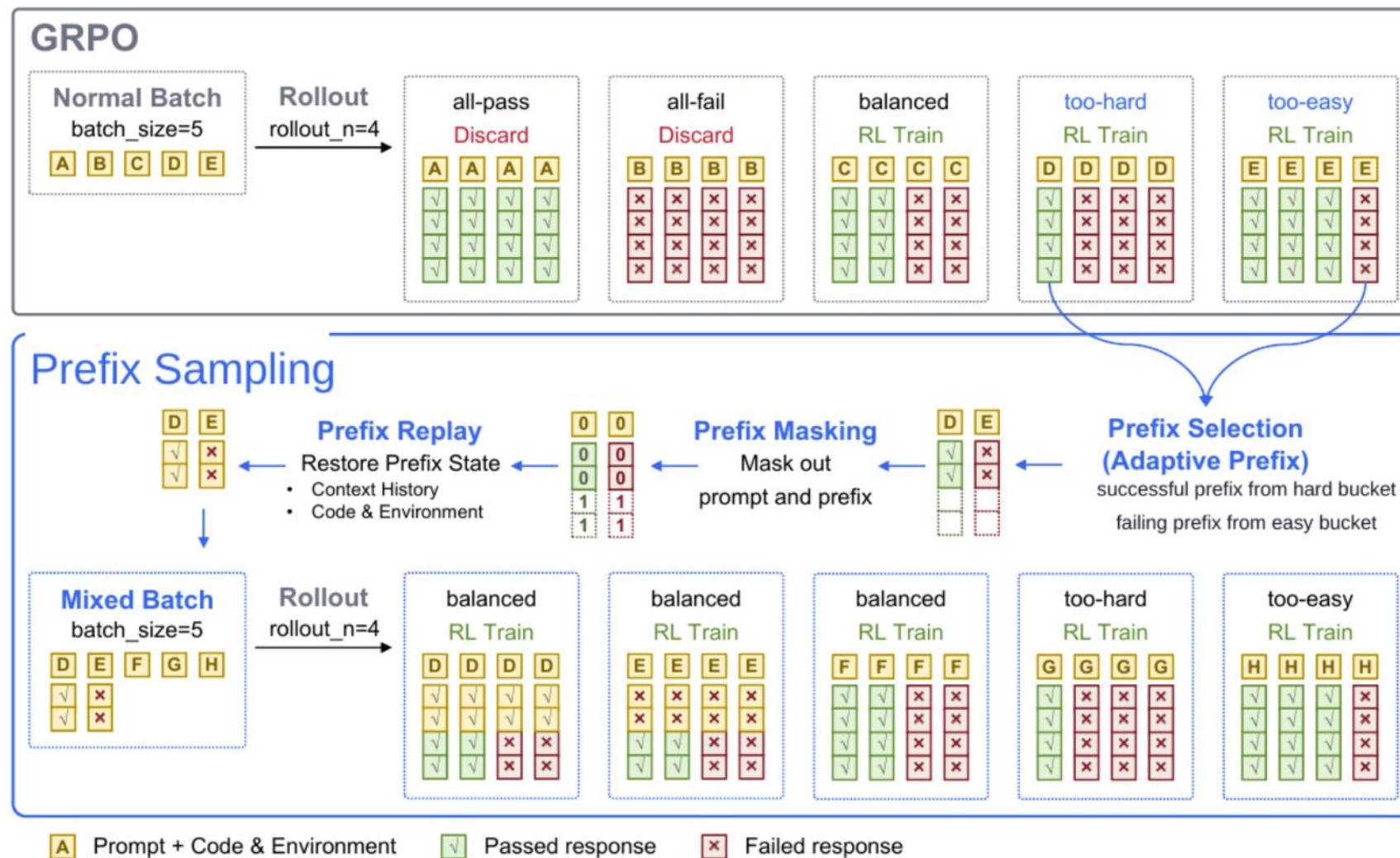
$$\bullet \left\{ \begin{array}{ll} H(p) = -p \log_2 p - (1-p) \log_2 (1-p), & \arg \max_p H(p) = 0.5 \\ S_N(p) = \Pr(0 < K < N) = 1 - p^N - (1-p)^N, & \arg \max_p S_N(p) = 0.5 \\ E_{\text{RLOO}}(k) = \frac{k(N-k)}{(N-1)^2}, & \arg \max_k E_{\text{RLOO}}(k) = N/2 \\ C(k) = k(N-k), & \arg \max_k C(k) = N/2 \end{array} \right.$$

- 单条 rollout 的成功概率为 p , 成功数为 k
- 熵 rollout 结果携带的信息量
- 组过滤存活率 有成功有失败
- RLOO 优势能量 留一法 baseline 是组内其他的平均奖励

成功: $A_i = 1 - \frac{k-1}{N-1} = \frac{N-k}{N-1}$; 失败: $A_i = 0 - \frac{k}{N-1} = -\frac{k}{N-1}$ 取平方平均

- 成败对比对数 一组回答里能凑出多少对

前缀采样



通过数距离 $|k-4|$

